BenchFuzz: Lockstep Differential Fuzzing, Hierarchical Delta Debugging and Guarded Repair for RISC-V RTL

A Controlled Study on PicoRV32 and a Roadmap to FPGA Acceleration

Subodh Satish Dalvi
School of Electrical and Computer Engineering, The University of Sydney
Draft v0.4 — October 2026

***Abstract*—Processor verification spends most of its effort on generating stimulus, deciding whether the design misbehaved, and turning a long failing test into something a designer can act on. Recent LLM-guided fuzzers such as ChipFuzzer report large gains in condition coverage and bug detection, but they evaluate coverage and bug finding without minimisation or repair, on different designs, metrics and budgets, which makes their numbers hard to transfer. We present BenchFuzz, a role-separated differential verification loop for RISC-V cores, and a controlled study of it on PicoRV32. An immutable Python RV32IM reference and Verilator-compiled RTL are compared at every retirement through the RISC-V Formal Interface; failures are sliced with architectural state preloading and minimised by flat and hierarchical delta debugging; candidate RTL patches pass lint, reproducer, prefix-growth and whole-suite gates before acceptance. On unmodified YosysHQ PicoRV32 the oracle produces no false alarm in 150 of 150 control runs and settles the long-open `mem_addr` discrepancy as a presentation-convention gap. The same harness reproduces Cascade's PicoRV32 bug P6 (CVE-2023-34899) on the commit Cascade evaluated, both with a directed probe and with blind fuzzing in a median of 2 programs, and its mutation fuzzer exposes a residual specification deviation on current master: a `fence` with a non-zero reserved `rd` field writes zero to that register. Across 7 seeds per method, constrained-random and coverage-guided mutational fuzzing saturate every reachable RTL line of the RV32IM configuration within a few hundred programs; all 64 unreached line points are configuration-dead or outside the ISA scope. Blind random fuzzing detects 14 of the 15 injected bugs, three tiers from bit-level operator faults to an intermittent bus-protocol fault, while one literal mutation from the original brief is equivalent on every valid program. Delta debugging reduces fuzzer-found failures by a median 93.1% in length and 86.2% in cycles. Positive and negative controls confirm that the repair gates accept correct patches and reject wrong ones. A spectrum-ranked template-repair baseline without an LLM produces gate-passing patches for 9 of 14 observable bugs, but whole-core formal equivalence certifies only 7: one accepted patch is wrong for rare operands and another changes an output the lockstep oracle never observes, although both passed every regression and held-out test. We tabulate the published results of ChipFuzzer, Cascade, TheHuzz, TurboFuzz, BMCFuzz and Encarsia with their sources, state precisely which comparisons are and are not meaningful, and give a measured-resource roadmap for moving the loop onto a Zynq UltraScale+ SoC following TurboFuzz.**

***Index Terms*—hardware fuzzing, RISC-V, differential testing, delta debugging, automated program repair, LLM, FPGA**

I. Introduction

Pre-silicon verification is the most expensive phase of processor design, often cited at over 70% of project effort [1]. Hardware fuzzers [7], [8], [3], [2] generate instruction streams, run them on RTL and compare against an ISA model. LLM-based generators [1] now steer generation toward uncovered code, and FPGA-resident fuzzers [4] remove the simulation bottleneck. Three gaps motivate this work.

(i) Oracle soundness is rarely measured. A differential oracle that disagrees with correct RTL for a convention reason (for example, how a load's address is presented) produces false alarms that the rest of the pipeline then tries to "debug". Our own earlier prototype did exactly that.

(ii) Detection is where most evaluations stop. Failing programs from fuzzers are long; Cascade prunes them [2], but minimisation and repair are rarely evaluated together with detection.

(iii) Published numbers do not transfer. Results are reported on different cores, coverage definitions (condition, mux-toggle, register-toggle), budgets and reference simulators, so a table that places them side by side with a new tool can mislead.

Contributions:

- A **role-separated architecture** (Sec. III) in which only one component may write the RTL, the reference model is immutable, and every LLM output passes through a validator or a patch gate.
- A **measured control on unmodified upstream RTL** (Sec. VII-A): zero false alarms on YosysHQ master and on the commit used by Cascade, and a resolution of the `mem_addr` 2019/2016 discrepancy as a convention gap.
- A **reproduction of a published bug**, Cascade P6, on the same DUT commit, and a **residual FENCE deviation** on current upstream master found by mutational fuzzing (Sec. VII-B).
- A **15-bug, three-tier injection suite** with observability checks, including a mutant that is equivalent on all valid programs and an intermittent bus fault (Sec. V).
- **Head-to-head coverage and time-to-bug benchmarks** with seven to ten seeds, confidence intervals and Mann–

Whitney tests on one harness (Secs. VII-C–VII-D), and a classification of everything left uncovered.

- **Hierarchical delta debugging** over basic blocks and instructions, evaluated against flat ddmin on fuzzer-found failures with a 1-minimality check (Sec. VII-E).
- **Guard validation for LLM repair** with positive and negative controls, a CirFix-style template baseline ranked by Ochiai suspiciousness, **formal certification** of accepted patches that exposes a gate-passing, held-out-passing but wrong patch, and an independent-sample pass@k harness (Sec. VII-F).
- A **unified co-coverage metric** with explicit weights and a demonstration of why components must be reported (Sec. VII-G); a **sourced comparison** with six published systems (Sec. VIII); and an **FPGA roadmap** with measured resource figures (Sec. IX).

II. Background and Related Work

Processor fuzzing: RFUZZ [7] brought coverage-guided mutation to RTL; DifuzzRTL [8] added control-register coverage; TheHuzz [3] used golden-model comparison and reached coverage 1.98× faster than random regression and 3.33× faster than DifuzzRTL on four cores, finding 11 bugs. Cascade [2] generates long programs whose control flow is entangled with their data flow so that a correct CPU always terminates; it reports the same coverage 28.2× and 97× faster than TheHuzz and DifuzzRTL and 37 new bugs (29 CVEs) in five of six cores, among them six in PicoRV32 (CSR, counter and `fence` handling). BMCFuzz [5] combines bounded model checking with fuzzing. Encarsia [6] injects realistic bugs into CPU netlists to evaluate fuzzers fairly, one bug per design copy.

LLM-guided fuzzing: ChipFuzzer [1] (preprint) selects uncovered RTL lines, retrieves the seed with the most similar control-flow path, asks an LLM for an assembly test with a discrepancy analysis, and corrects syntax and semantics in at most five rounds each; a bug-guided stage mines 128 historical bug fixes. On RocketCore, BOOM and CVA6 it reports condition coverage 9.1, 1.2 and 7.0 points above the strongest baseline after 24 h, and 61.1% of 90 Encarsia bugs against 36.7–40%. Its ablation is instructive: an LLM without guidance reaches 61.4% condition coverage and finds no bug.

Acceleration: TurboFuzz [4] (preprint) moves generation, execution, checking and coverage onto one Zynq UltraScale+ SoC and reports 309,676 executed instructions per second against 2,489 for Cascade and 728 for FPGA-assisted DifuzzRTL.

Debugging and repair: Delta debugging [9] minimises failure-inducing inputs; HDD [10] applies it level by level over a tree. Spectrum-based fault localisation (Ochiai [11]) ranks code by correlation with failing runs. CirFix [12] repairs Verilog by template search guided by simulation. pass@k [13] measures sampled code generation.

III. System Architecture

Figure 1 shows the loop. Table I lists what each role may read and write. The boundaries are enforced in code, not by convention: the reference model has no dependency on the harness; generators can only emit programs that the validator accepts; patches are spliced only into the line ranges the localiser offered; and acceptance requires that the entire regression suite match after the patch with no previously passing program failing.

TABLE I
Role-separated workflow boundaries.

Role	May read	May write
Generators (random, mutation, LLM)	coverage summary, uncovered lines, corpus	programs (validated before use)
Reference model	program	retirement records only
RTL simulator	mutable RTL copy, program	RVFI trace, coverage
Scoreboard	both traces	verdict, first mismatch
Delta debugger	failing program, verdict oracle	reproducers
Localiser / SBFL	RTL text, coverage spectra	candidate blocks
Repair agent	offered blocks, reproducer, signature	block patches (text)
Gates	candidate RTL, suite	accept / reject; checkpoint in `logs/checkpoints/`

Oracle: For program p let r_i^{ISS} and r_i^{RTL} be the i-th retirement records (PC before and after, instruction, destination register and value, memory address, masks and data). The verdict is

$$\mathrm{ok}(p) \iff |R^{\mathrm{RTL}}| = |R^{\mathrm{ISS}}| \ \wedge\ \forall i:\ \pi\big(r_i^{\mathrm{ISS}}\big) = r_i^{\mathrm{RTL}}, \quad (1)$$

where π applies the DUT's presentation convention. For PicoRV32 (riscv-formal [14] `ALIGNED_MEM`) π replaces the architectural address a by $a \wedge \neg 3$ and leaves the byte lane to the masks. Missing and extra retirements are failures, so a core that traps early cannot pass by default. The DUT is built with `ENABLE_FAST_MUL`, `ENABLE_DIV` and `REGS_INIT_ZERO`; the validator accepts exactly RV32IM plus `fence`.

IV. Metrics

RTL coverage: Let $P = P_{\mathrm{line}} \cup P_{\mathrm{tog}} \cup P_{\mathrm{br}}$ be the Verilator coverage points of the build and $H(p) \subseteq P$ those hit by p. After k programs

$$C_k = \frac{\left|\bigcup_{j\le k} H(p_j)\right|}{|P|}, \qquad C_k^x = \frac{\left|\bigcup_{j\le k} H(p_j) \cap P_x\right|}{|P_x|}. \quad (2)$$

Reference-model statement coverage: With S the executable statements of the reference model and $S(p)$ those executed, $C_{\mathrm{ISS}} = |\bigcup_j S(p_j)|/|S|$ ($|S| = 118$). Our reference is Python, so this replaces gcov on a C++ ISS.

Functional coverage: Five covergroups $G_1..G_5$ with bins B_g (49 mnemonics; 12 branch-type×outcome bins; 20 load/store size×byte-offset bins; 12 RV32M corner cases such as division by zero and signed overflow; 7 register-value patterns): $C_{\mathrm{func}} = \frac{1}{5}\sum_g |\mathrm{hit}_g|/|B_g|$.

Unified co-coverage:

$$C_U = w_L C^{\mathrm{line}} + w_T C^{\mathrm{tog}} + w_S C_{\mathrm{ISS}} + w_F C_{\mathrm{func}}, \quad \textstyle\sum w = 1, \quad (3)$$

with equal weights $(0.25, 0.25, 0.25, 0.25)$ unless stated. C_U is a monitoring aggregate, not a validated predictor of bug finding; we always report its components.

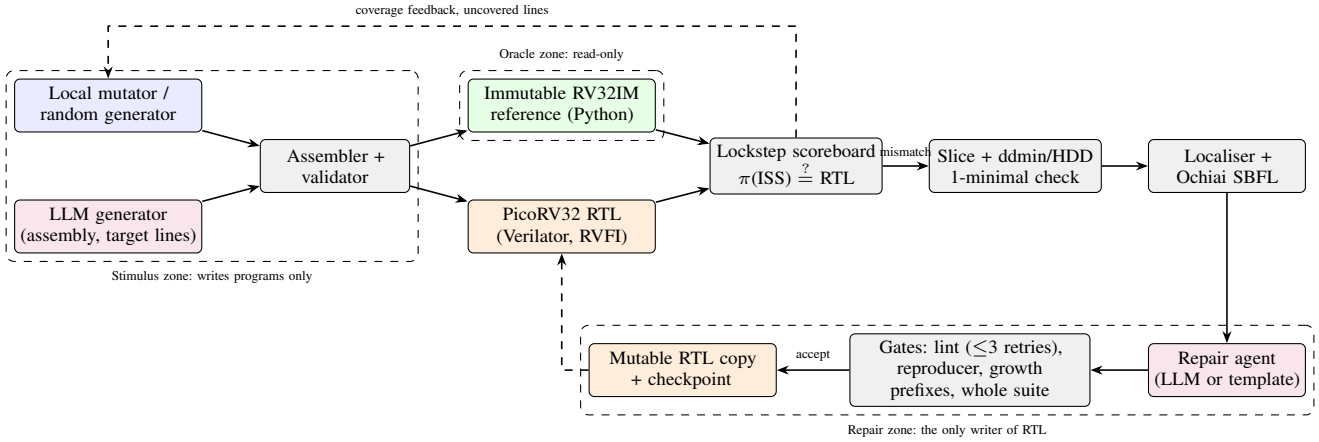


Fig. 1. BenchFuzz workflow and role boundaries. Dashed boxes are trust zones; solid arrows carry data, dashed arrows feedback.

Plateau and efficiency: Following ChipFuzzer, a run has plateaued at k when $C_k - C_{k-100} < 0.1$ percentage points. With C^* the best final coverage over all runs, $N_\rho = \min\{k : C_k \geq \rho C^*\}$ is the number of programs needed to reach a relative threshold ρ.

Time-to-bug: N_{bug} is the index of the first program with $\neg\text{ok}(p)$ on the buggy RTL; we also report seconds, excluding the one-off Verilator build.

Minimisation: For a failing p and its minimised p^*: $R_{\text{len}} = 1 - |p^*|/|p|$ (instructions) and $R_{\text{cyc}} = 1 - \text{cyc}(p^*)/\text{cyc}(p)$; p^* is 1-minimal if removing any single unprotected instruction makes it pass.

Suspiciousness: For RTL line ℓ with e_f failing and e_p passing runs executing it out of F failing runs, $\text{Ochiai}(\ell) = e_f/\sqrt{F(e_f + e_p)}$.

Repair: With n independent samples and c successes, $\text{pass@}k = 1 - \binom{n-c}{k}/\binom{n}{k}$ [13].

V. Bug Suite

Each bug is a list of exact-text edits applied to a fresh copy of the golden RTL (multi-edit bugs are applied in order; markers must be unique). Table II lists the suite. Tier L1 contains signal and operator faults, L2 control and block faults, and L3 system and protocol faults. Five legacy bugs come from the prototype; they contain conspicuous constants (for instance `latched_rd == 5`) that an LLM can spot by reading, so ten plain mutations were added. Three items of the original brief needed refinement (details in Appendix A): the only AND operators in `picorv32_pcpi_fast_mul` are logical; mutating that decode term gives a mutant that is equivalent on valid programs (Sec. VII-D), so we also add an observable signedness fault inside the same module and keep the bitwise AND→OR in the ALU. `mem_ready` is a testbench input, so the handshake fault is placed in the core's reaction to it, in a deterministic and an intermittent (timing-gated) variant. Interrupt trap flags are unobservable with `ENABLE_IRQ=0` and no interrupt model in the reference, so an uncleared `latched_branch` flag substitutes.

VI. Experimental Setup

All experiments ran in WSL2 Ubuntu on an 8-core x86-64 laptop with 15 GB RAM, Verilator 5.032 and Yosys 0.52, with the DUT at YosysHQ master (`ef203c2`) unless stated. Generators: *Random* (constrained-random RV32IM programs of 20 body instructions over a pool of interesting values), *Cov.-guided mutation* (an AFL-style corpus loop that keeps programs adding coverage and mutates by bit flips outside the opcode, swaps, splices, immediate rewrites and fresh segments), and *Hybrid* (mutation until the plateau rule fires, then random, the equal-budget control for ChipFuzzer's plateau switch). These are re-implementations of strategies, not the published tools. Budgets are counted in validated programs, the testcase unit of ChipFuzzer's Table 2, so results do not depend on machine load: 10,000 programs per coverage run (7 seeds per method) and 1,000 per time-to-bug run (ten seeds per bug and method). Between seven and twelve processes shared the eight cores, so seconds and per-process throughput in the benchmark tables are measured under load; a separate single-process run on the idle machine gives the throughput figure of Sec. VIII.

LLM availability: No LLM key was available in this evaluation environment. LLM generation and LLM repair (pass@k) are implemented and unit-tested with a scripted model; their measurement is future work, and every LLM-dependent statement below refers to the archived 552-iteration Gemini campaign (Run C) of the previous version.

VII. Results

A. RQ1: Is the oracle sound on unmodified RTL?

Table III shows 150/150 matching runs (1,599 retired instructions) on YosysHQ master and 150/150 on `f00a88c`. At scale, the 210,000 programs of the coverage benchmark (Sec. VII-C), all run on master, raised 0 mismatches. Our previous "golden" copy was *not* pristine: it differed from upstream in the `REGS_INIT_ZERO` default and in a simplified `mem_la_read`. Both edits are behaviourally inert in this configuration (identical E0 results), but the golden copy is now byte-identical to upstream.

TABLE II
E1: BUG SUITE, TARGETED STIMULUS, MINIMISATION OF THE STIMULUS AND LOCALISATION (HIT = AN OFFERED BLOCK CONTAINS THE EDIT).

Bug	Tier	Failing field / kind	Stim.	Min.	Loc.	Hit
`approx_multiplier`	L1	field_mismatch	6	4	multiplier	✓
`alu_operator_xor_or`	L1	field_mismatch	4	3	alu	✓
`branch_inversion_blt`	L2	field_mismatch	6	3	branch	✓
`writeback_forwarding_drop`	L2	field_mismatch	5	3	alu	✓
`mem_handshake_drop`	L3	field_mismatch	6	2	memory	✓
`and_to_or_alu`	L1	field_mismatch	5	3	alu	✓
`fast_mul_decode_and_to_or`	–	not observable (equivalent mutant)				
`fast_mul_rs2_signed_widened`	L1	field_mismatch	5	4	multiplier	✓
`mem_addr_bit2_stuck`	L1	field_mismatch	7	2	memory	✓
`slt_signed_as_unsigned`	L1	field_mismatch	6	3	alu	✓
`beq_inverted`	L2	field_mismatch	6	2	branch	✓
`lui_auipc_rd_x0`	L2	field_mismatch	5	2	fetch_launch	✓
`store_after_store_dropped`	L3	field_mismatch	7	2	memory	✓
`store_after_store_dropped_intermittent`	L3	field_mismatch	27	6	memory	✓
`stale_latched_branch`	L3	field_mismatch	8	3	branch	✓

TABLE III
E0 CONTROL RUNS (47+ PROGRAMS × MEMORY DELAYS 0, 1, 3; SEEDS, MUTATED SEEDS, RV32M EDGE CASES, EVERY BUG STIMULUS, ALIGNMENT PROBE).

RTL	Commit	Matched runs	Retired insns	Aligned conv.	FENCE (P6)	FENCE rd≠0 (x_2=123?)
YosysHQ master	`ef203c2`	150/150	1,599	yes	matches	x2=0
f00a88c (Cascade)	`f00a88c`	150/150	1,599	yes	trap	trap
golden copy	–	150/150	1,599	yes	matches	x2=0

The `mem_addr` question: The alignment probe executes `lb x3, 1(x1)`. Under the architectural convention the reference expects address 257 and unmodified upstream RTL reports 256; under π the run matches. The earlier 2019-versus-2016 discrepancy therefore occurs on pristine upstream and is a convention gap, not a defect of our mutable copy. Toggle coverage corroborates it: `mem_la_addr[1:0]` never toggles in any run.

B. Reproducing a published bug: Cascade P6

Cascade reports that PicoRV32 `f00a88c` raises a spurious exception on `fence` (P6, CVE-2023-34899); upstream fixed the decoder later. After adding `fence` as an architectural no-op to the reference, the E0 probe diverges on `f00a88c` (the core traps before retiring the `fence`) and matches on master. With `fence` emitted at 2% per slot, blind fuzzing hit P6 in 10/10 random seeds (median 2 programs, 0.15 s) and 10/10 mutation seeds (median 4 programs). P1–P5 involve CSRs and counters, outside our RV32IM scope.

A residual FENCE deviation on master: Running the same configuration on master as a control, the random generator, which emits only canonical `fence` words, raised no alarm in 30,000 programs. The mutation fuzzer, which flips non-opcode bits, hit a mismatch in 9/10 seeds (median 872 programs), and 9 of these first mismatches are a `fence` whose reserved `rd` field is non-zero. The directed probe (Table III, last column) shows the consequence: after `x1=123; fence` with $rd{=}x1$, master holds x_1=0, i.e. the core performs an architecturally visible write-back of zero. The unprivileged specification states that base implementations shall ignore the reserved `rd` and `rs1` fields of FENCE. Upstream issue #293 reported that these fields are decoded and was closed on the grounds that decoding them has no side effect; our result shows that it has one. Standard toolchains always emit zero fields, so the practical impact is low, but it is a specification deviation that survived the P6 fix and that only a mutational generator found.

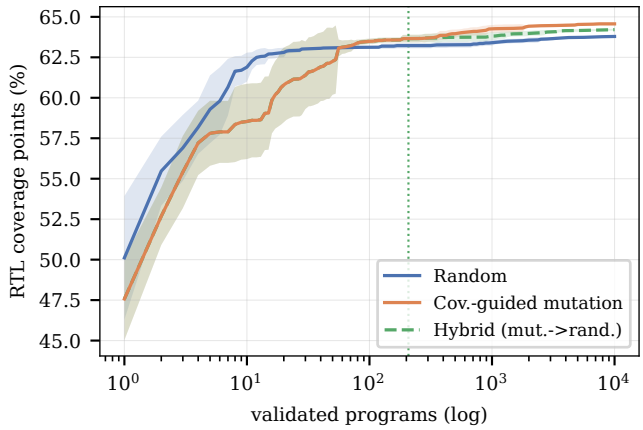


Fig. 2. Cumulative RTL coverage (line, toggle and branch points, mean and 95% CI over seeds). Dotted: median plateau switch of Hybrid.

TABLE IV
COVERAGE BENCHMARK AT $N = 10,000$ PROGRAMS ($|P| = 4,703$ POINTS). $N_{0.90}$, $N_{0.99}$: MEAN PROGRAMS TO REACH 90% AND 99% OF $C^* = 64.62\%$; PLATEAU BY THE CHIPFUZZER RULE; EXECUTED INSTRUCTIONS PER SECOND PER PROCESS.

Method	Cov.@N (%) mean±CI	90%C^* progs	99%C^* progs	Plateau progs	Insn/s
Random	63.79±0.00	5	–	133	186
Cov.-guided mutation	64.57±0.08	14	648	227	371
Hybrid (mut.→rand.)	64.21±0.06	14	1,998	227	132

C. RQ2: Coverage, plateaus and efficiency

Figure 2 and Table IV summarise the coverage runs. Random reaches 63.79±0.00% and coverage-guided mutation 64.57±0.08% (Mann–Whitney $p = 0.001$); Hybrid reaches

TABLE V
WHY COVERAGE PLATEAUS: UNREACHED LINE-COVERAGE REGIONS AFTER ALL RANDOM RUNS, BY CAUSE.

Cause	Regions	Source lines
Compressed ISA (COMPRESSED_ISA=0)	28	40
Interrupts (ENABLE_IRQ=0)	14	14
Counter CSRs (outside RV32IM scope)	2	8
ld_rs2 state (unused, ENABLE_REGS_DUALPORT=1)	4	6
ALU options (BARREL_SHIFTER, TWO_CYCLE_*)	3	5
External PCPI (ENABLE_PCPI=0)	1	1
Trap on illegal instruction (excluded by validator)	1	1

64.21±0.06% ($p = 0.001$ against mutation). All methods meet the ChipFuzzer plateau rule early (random after 133 programs, mutation after 227 on average), yet mutation keeps adding points slowly afterwards, so Hybrid, which abandons mutation when the rule fires, ends below mutation: on this DUT the rule fires prematurely, and an LLM switched in at that point would be competing against a generator that has not actually stalled. The 39 points that mutation reaches and random never does are all toggle bits of high address, PC, `rvfi_order` and cycle/instruction-counter bits: mutation's splices lengthen programs and its immediate rewrites widen the address range, whereas random programs have a fixed length and a 2 KiB data window (random reaches 0 points that mutation misses). The gain is therefore a program-shape effect, not new control behaviour. The plateau is a ceiling, not a weakness of the generators: Table V classifies the 64 of 180 line points that no run reached, in 53 regions. Every region is compressed-ISA, interrupt, counter-CSR, ALU-option, external-PCPI or unused-state logic, or the illegal-instruction trap that the validator excludes (0 unexplained). For this configuration the reachable line coverage is therefore 100%, and further coverage requires widening the ISA scope or the build, not smarter generation. This mirrors Cascade's observation that PicoRV32's 172 mux-select points saturate in about 100 s [2], and it is the main reason why ChipFuzzer-style coverage gains cannot be expected on this DUT.

D. RQ3: Which bugs does blind fuzzing find, and how fast?

Figure 3 and Tables VI–VII report detection. Random detects 14 and mutation 13 of the 15 bugs in at least one seed (13 and 13 in every seed; per-run detection rates 88.0% and 86.7%). The literal AND→OR mutation in the multiplier's decode is never detected, by the targeted stimulus or by any fuzzing run. By inspection it is equivalent on every valid RV32IM program: decoded instructions never reach the PCPI bus, and `funct7=1` R-type instructions already satisfy both terms. It differs only for encodings the core does not decode, which PicoRV32 forwards to PCPI: with the mutation the multiplier would claim an illegal instruction with `funct7=1` and retire it instead of trapping. Our validator excludes illegal encodings, so the mutant is out of reach by construction; the whole-core equivalence check did not terminate within 600 s, so we do not claim formal equivalence.

Apart from that mutant, only `approx_multiplier` resists: its trigger is a conjunction of two operand nibbles of a multiplication (about one MUL in 256 even with uniformly random operands), and random found it in 2 of 10 seeds, mutation in none. Every other bug is detected in every seed, with median latencies of one to three programs in each tier (Table VII). The tier of a fault, its structural level, therefore does not predict how hard it is to trigger; the specificity of its activation condition does. Even the legacy `writeback_forwarding_drop`, whose trigger names register x5, is hit by the first program because the random preamble writes and re-reads every low register. Coverage guidance does not help detection here: mutation is never faster than random by more than a program and is slower on the signedness fault in the multiplier (median 52 against 10.5 programs) and on the intermittent store fault (21 against 11), because its corpus concentrates on programs that toggled new bits rather than on fresh operand combinations.

E. RQ4: How small do failures get, at what cost?

We minimised 80 fuzzer-found failures over 14 bugs (Table VIII). The median failing program of 36 instructions shrinks to 3 ($R_{\text{len}} = 93.1\%$, $R_{\text{cyc}} = 86.2\%$). The state-preloading slice kept the failure in 67 cases; otherwise minimisation starts from the original program, which matters for faults in micro-architectural state (`stale_latched_branch`). HDD and flat ddmin reached the same size in 73 cases and HDD a smaller one in 4; HDD needed fewer oracle tests in 29 cases (medians 7 versus 6, Fig. 4). 100% of HDD and 100% of ddmin results were 1-minimal. The hierarchy does not pay off here because the state-preloading slice does most of the work: it already cuts the failing program to the window before the failing retirement (median 19 instructions), and at that size flat ddmin needs only about six simulations while HDD's extra basic-block level is overhead. HDD is designed for long, structured inputs; fuzzer failures on an in-order core that diverge within tens of instructions are not such inputs. The 4 cases in which HDD reached a strictly smaller reproducer than flat ddmin are all failures that depend on micro-architectural or timing state (`stale_latched_branch` and the intermittent store fault); this is consistent with HDD paying off for long Cascade-style programs whose failure needs distant state set-up that the slice cannot preload, which we have not evaluated.

F. RQ5: Do the repair gates work, and how far does a non-LLM baseline get?

To evaluate the gates independently of any model we drive the unchanged pipeline (slice, ddmin, localisation, patch parsing, block-range check, lint with feedback retries, reproducer, prefix growth) through a scripted stand-in for the LLM client, then apply the campaign's acceptance rule (Table IX). The *oracle* returns the golden text of the hunks inside the offered blocks and was accepted for 14/14 bugs, i.e. localisation and gates never block a correct fix. A *lint-broken* oracle patch was rejected by Verilator and recovered on the retry for 14/14. The *no-op* patch was rejected by the reproducer gate for 14/14. The template baseline [12] had a patch accepted by all gates for 9/14 bugs within 30 candidates (median 15 tried); 9 of these also passed all held-out programs and 4 were textually the

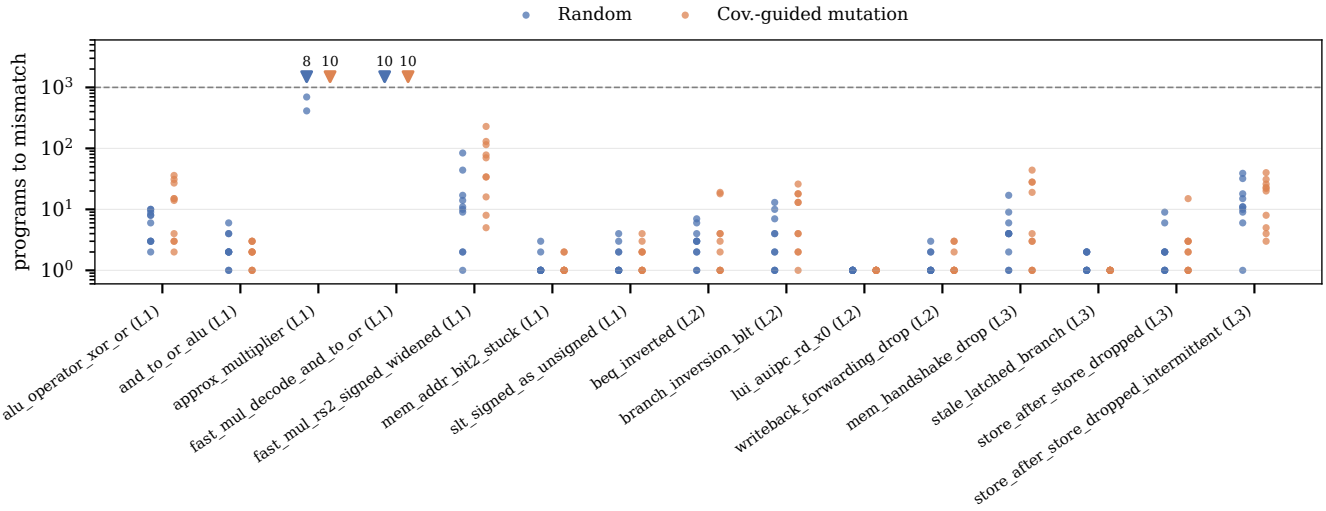


Fig. 3. Programs to first lockstep mismatch per bug and seed (log scale). Triangles: seeds that did not detect the bug within the budget (dashed line); the number is the count of such seeds.

TABLE VI
TIME-TO-BUG, BLIND FUZZING (NO TARGETED STIMULUS), TEN SEEDS, BUDGET 1,000 PROGRAMS.

		Random		Cov.-guided mut.	
Bug	Tier	Det.	Progs (med.)	Det.	Progs (med.)
`alu_operator_xor_or`	L1	10/10	7	10/10	14
`and_to_or_alu`	L1	10/10	2	10/10	2
`approx_multiplier`	L1	2/10	554	0/10	–
`fast_mul_decode_and_to_or`	L1	0/10	–	0/10	–
`fast_mul_rs2_signed_widened`	L1	10/10	10	10/10	52
`mem_addr_bit2_stuck`	L1	10/10	1	10/10	1
`slt_signed_as_unsigned`	L1	10/10	1	10/10	2
`beq_inverted`	L2	10/10	3	10/10	2
`branch_inversion_blt`	L2	10/10	3	10/10	8
`lui_auipc_rd_x0`	L2	10/10	1	10/10	1
`writeback_forwarding_drop`	L2	10/10	1	10/10	1
`mem_handshake_drop`	L3	10/10	4	10/10	4
`stale_latched_branch`	L3	10/10	1	10/10	1
`store_after_store_dropped`	L3	10/10	2	10/10	2
`store_after_store_dropped_intermittent`	L3	10/10	11	10/10	21

TABLE VII
DETECTION LATENCY BY TIER (MEDIAN OVER ALL DETECTING RUNS).

Tier	Method	Detections	Progs (median)	Seconds (median)
L1	Random	52	2	0.94
L1	Cov.-guided mutation	50	2	1.00
L2	Random	40	1	0.50
L2	Cov.-guided mutation	40	1	0.55
L3	Random	40	2	0.75
L3	Cov.-guided mutation	40	3	0.62

golden fix. Across all candidates the gates rejected 0 at lint, 269 at the reproducer, 0 at prefix growth and 5 at the suite.

Plausible is not correct: Gate acceptance and held-out agreement are both simulation evidence, and they can be wrong together. For `approx_multiplier` the search inverted one comparison of the injected trigger (`rs1[11:8]==9` → `!=9`) instead of removing the XOR term. The patch passes the reproducer, the whole suite and all 400 held-out runs, because random programs almost never multiply by an operand with `rs2[7:4]=0xC`; the directed witness `0x100×0xC0` returns 49184 instead of 49152. We therefore certify every accepted patch formally: Yosys structural and inductive equivalence of the whole core against the golden RTL in the harness configuration (proves golden against itself in about 2 min; 300 s budget). 7 accepted template patches are certified (identical or proven equivalent) and 2 are not proven; 1 of the latter is refuted by a directed witness although it passed the held-out set. The other, for `mem_handshake_drop`, is a subtler case: 2,400 of the core's equivalence points are proven equal and the single unproven one is the top-level output `mem_la_write`. The patch turned an `&&` into `||` at a precedence boundary, so the look-ahead write strobe is asserted whenever the bus is idle; the native bus is unaffected because the state machine clears `mem_wstrb` on the same edge, so no retirement record can show the difference (fuzzing the patched core for 18,000 further programs raised 0 mismatches). A system that uses

TABLE VIII

DELTA DEBUGGING OF FUZZER-FOUND FAILURES (MEDIANS PER BUG): STATIC LENGTH, SLICE, FLAT DDMIN, HDD, ORACLE TESTS, CYCLES.

Bug	n	Orig.	Slice	ddmin	HDD	Tests dd/HDD	Cycles orig.→min.
alu_operator_xor_or	6	34	18	3.5	3.5	12/14	168→24
and_to_or_alu	6	34	20	3.0	3.0	6/8	160→23
approx_multiplier	2	36	20	5.5	5.5	27/28	260→35
fast_mul_rs2_signed_widened	6	62	20	4.0	4.0	16/18	314→29
mem_addr_bit2_stuck	6	34	19	2.0	2.0	5/1	162→22
slt_signed_as_unsigned	6	35	20	3.0	3.0	6/10	165→22
beq_inverted	6	34	22	2.0	2.0	5/1	180→20
branch_inversion_blt	6	50	22	3.0	3.0	9/10	242→23
lui_auipc_rd_x0	6	35	2	2.0	2.0	0/0	150→20
writeback_forwarding_drop	6	34	13	3.0	3.0	4/5	159→23
mem_handshake_drop	6	76	18	2.0	2.0	5/1	345→22
stale_latched_branch	6	35	18	3.0	2.5	8/4	132→20
store_after_store_dropped	6	36	18	4.0	4.0	15/18	138→30
store_after_store_dropped_intermittent	6	60	18	19.0	18.0	66/84	300→83

TABLE IX

GUARDED REPAIR ON THE 14 OBSERVABLE BUGS. CONTROLS DRIVE THE UNCHANGED PIPELINE THROUGH A SCRIPTED MODEL; TEMPLATE REPAIR SEARCHES GENERIC SINGLE-LINE OPERATORS RANKED BY OCHIAI; HELD-OUT: 200 UNSEEN RANDOM PROGRAMS × DELAYS 0 AND 3; FORMAL: YOSYS WHOLE-CORE EQUIVALENCE WITH THE GOLDEN RTL (IDENT. = TEXTUALLY IDENTICAL, N.P. = NOT PROVEN; † REFUTED BY A DIRECTED WITNESS, ‡ DIFFERS ONLY AT THE MEM_LA_WRITE OUTPUT).

	Guard controls			SBFL template repair (budget 30)			
Bug	Oracle	Lint→retry	No-op	Acc.	Tried	Held-out	Formal
alu_operator_xor_or	✓	✓	rej.	✓	15	400/400	proven
and_to_or_alu	✓	✓	rej.	✓	14	400/400	ident.
approx_multiplier	✓	✓	rej.	✓	27	400/400	n.p.†
beq_inverted	✓	✓	rej.	✓	15	400/400	ident.
branch_inversion_blt	✓	✓	rej.	×	30	–	–
fast_mul_rs2_signed_widened	✓	✓	rej.	✓	11	400/400	ident.
lui_auipc_rd_x0	✓	✓	rej.	×	30	–	–
mem_addr_bit2_stuck	✓	✓	rej.	✓	18	400/400	ident.
mem_handshake_drop	✓	✓	rej.	✓	6	400/400	n.p.‡
slt_signed_as_unsigned	✓	✓	rej.	×	30	–	–
stale_latched_branch	✓	✓	rej.	×	30	–	–
store_after_store_dropped	✓	✓	rej.	✓	16	400/400	proven
store_after_store_dropped_intermittent	✓	✓	rej.	✓	16	400/400	proven
writeback_forwarding_drop	✓	✓	rej.	×	30	–	–

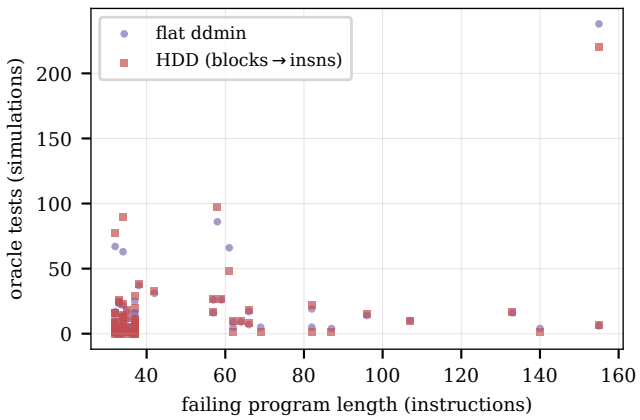


Fig. 4. Oracle cost of minimisation: simulations used by flat ddmin and by HDD against the length of the failing program.

PicoRV32's look-ahead interface would see spurious write requests. Repair results that rely on test suites, or on an oracle that observes only architectural state, should be read with both cases in mind.

Of the five bugs the template search did not repair, stale_latched_branch needs a deleted statement re-inserted, which no single-line operator can do. For the other four (branch_inversion_blt, writeback_forwarding_drop, lui_auipc_rd_x0: a ternary collapse; slt_signed_as_unsigned: a $signed wrap) the correct template exists, but the edit ranked beyond the 30-candidate budget, because Ochiai ranks lines, not edits. Verilator records no line coverage for continuous assignments, so those lines cannot be ranked by evidence at all (we let them inherit their block's highest score). Choosing the edit, not only the line, is where an LLM is expected to add value.

The archived Gemini campaign (Run C: 552 iterations, five legacy bugs) accepted 4 patches that together removed all five bugs, after ten rolled-back candidates; one accepted patch also removed a bug that the failing signature did not implicate, which we attribute to code reading of the conspicuous legacy constants. pass@1/pass@5 with independent samples (harness/repair_eval.py) await an LLM key.

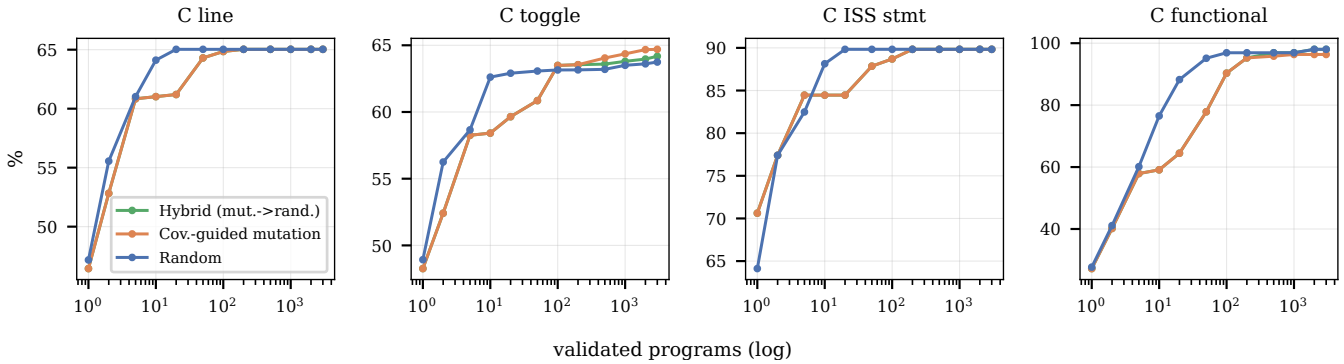


Fig. 5. Growth of the four C_U components (mean over seeds).

TABLE X
UNIFIED CO-COVERAGE COMPONENTS AT THE END OF THE RUNS SAVED WITH PROGRAMS (EQUAL WEIGHTS).

Method	C_{line}	C_{tog}	C_{ISS}	C_{func}	C_U
Random	65.03	63.74	89.83	98.02	79.15
Cov.-guided mutation	65.03	64.70	89.83	96.35	78.98
Hybrid (mut.→rand.)	65.03	64.18	89.83	98.02	79.26

G. RQ6: Unified coverage and instruction skew

Table X and Fig. 5 show three effects. (i) Line coverage is identical for every method and seed ($C_{\text{line}} = 65.03\%$), consistent with Sec. VII-C: every reachable line is covered. (ii) C_{ISS} saturates at 89.83% for every method; the remaining statements are the reference's `clone()` helper and error paths that valid programs cannot reach. (iii) The aggregate reverses the RTL ranking: mutation toggles more RTL bits than random (64.70% against 63.74%), but its corpus hits fewer functional bins (96.35% against 98.02%), so with equal weights C_U ranks random first (79.15% against 78.98%). The reference-side terms also lift C_U about 15 points above the RTL components without exercising any more hardware. The only functional bins never hit are fence, jalr (by no run of any method), i.e. instructions our generators never emit; the signed-overflow corner cases were hit by some runs but not all. We therefore report components and use C_U only for monitoring.

The archived Gemini campaign shows the skew the brief asked about: 57.5% of retired instructions were ALU-immediate against 5.5% stores, 8.2% branches and 2.0% jumps (Fig. 6), caused by the `li` preambles and by the local mutator. Of 133 Gemini expansion requests only 5 programs passed the validator, mostly because hand-encoded machine words were invalid; the assembler-based target-line generator (`harness/semantic_gen.py`) addresses that, but whether target-guided prompting beats blind mutation after the plateau remains open on this DUT, and Sec. VII-C suggests the reachable space leaves it little room.

VIII. Comparison with the State of the Art

Tables XI–XIII and Figs. 7 and 8 quote published results (test-case efficiency in Table XII, Encarsia detection in Table XIII);

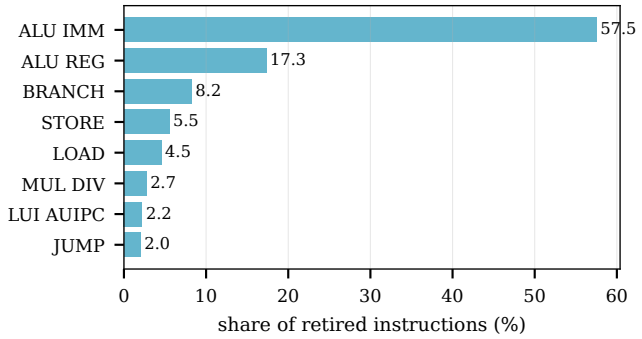


Fig. 6. Instruction-class skew in the archived 552-iteration Gemini campaign (Run C).

none was re-measured, and *the rows are not comparable with our measurements*: different cores (RocketCore, BOOM, CVA6 versus PicoRV32), coverage definitions (condition, mux- and register-toggle versus Verilator line/toggle/branch), budgets (24 h versus program counts) and references (Spike versus our Python model). Two numbers are comparable in kind: our reproduction of Cascade P6 on the same commit, and executed instructions per second as an order of magnitude.

Capability comparison: Relative to ChipFuzzer, BenchFuzz has a stronger oracle (every retirement instead of end state) and adds minimisation, localisation and gated repair; ChipFuzzer has path-similarity seed retrieval, structured discrepancy analysis, a bug-guided stage with historical priors and seed fusion, which we lack. Relative to Cascade, BenchFuzz checks every retirement instead of relying on program termination, at the cost of a reference model, and does not cover CSRs or privilege.

Throughput: Run alone on the idle machine, our loop executes 544 (random) and 1,114 (mutation, longer programs) instructions per second in one process (Fig. 9), below Cascade's 2,489 and two to three orders of magnitude below TurboFuzz. Each 30-instruction program costs one process launch, a Verilator coverage dump and a Python reference run; batching programs per simulation and a native reference would be the first optimisations, and the FPGA path below the decisive one.

TABLE XI
PUBLISHED SYSTEMS (PEER REVIEW STATUS AS OF OCTOBER 2026).

System	Venue	Reported result (quoted)
TheHuzz [3]	USENIX Sec. '22	coverage 1.98×/3.33× faster than random regression/DifuzzRTL; 11 bugs (8 new) on 4 cores
Cascade [2]	USENIX Sec. '24	same coverage 28.2×/97× faster than TheHuzz/DifuzzRTL; 37 bugs, 29 CVEs; PicoRV32 `f00a88c`: P1–P6
BMCFuzz [5]	IEEE '25	BMC + fuzzing; figures below are as reported by ChipFuzzer
Encarsia [6]	USENIX Sec. '25	injected-bug benchmark; 90 bugs (30 per core) as used by ChipFuzzer
TurboFuzz [4]	arXiv (preprint)	on-SoC fuzzing (XCZU19EG, Rocket @100 MHz): 309,676 insn/s; up to 2.23× coverage, 571× faster detection
ChipFuzzer [1]	arXiv (preprint)	cond. coverage +9.1/+1.2/+7.0 pp (Rocket/BOOM/CVA6, 24 h); Rocket 89.2% vs 80.06% (BMCFuzz); 61.1% of Encarsia bugs

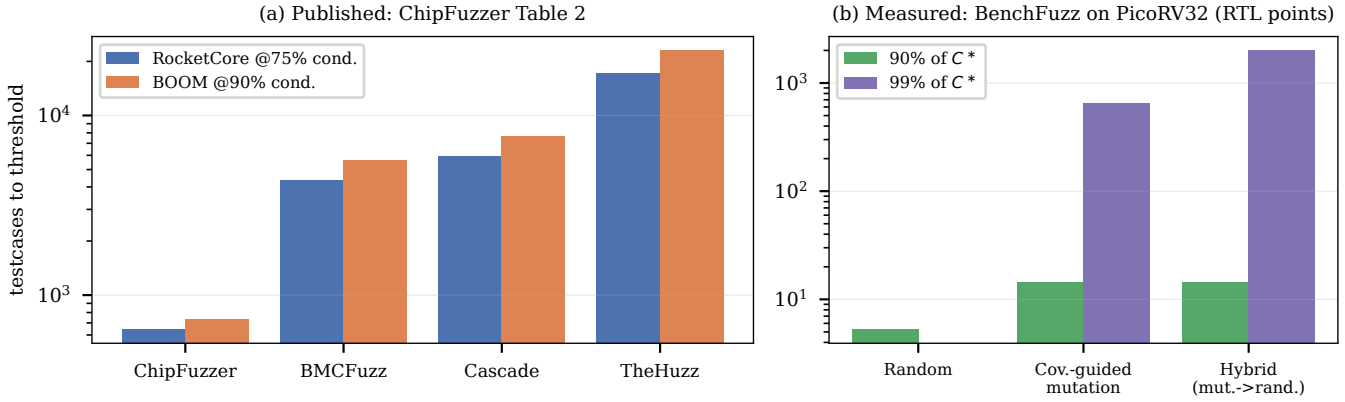


Fig. 7. Testcase efficiency. (a) Published, absolute condition-coverage thresholds on RocketCore and BOOM. (b) Measured on PicoRV32, relative thresholds of the best observed RTL coverage. The panels use different designs and metrics and must not be compared numerically; together they show that the hard part on large out-of-order cores (reaching the last conditions) does not exist on PicoRV32's RV32IM configuration.

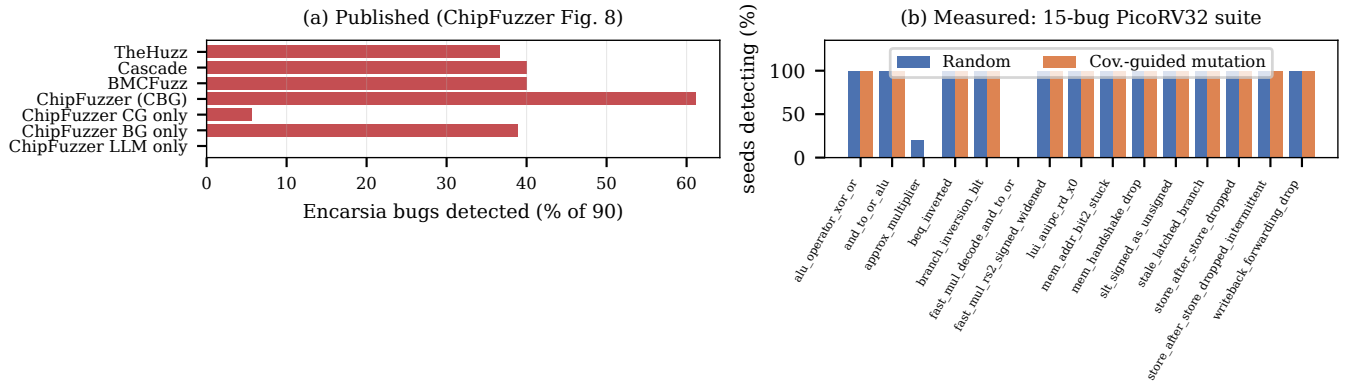


Fig. 8. Bug detection. (a) Published Encarsia detection rates. (b) Our per-bug detection rate across ten seeds. Different bug corpora and designs.

TABLE XII
TESTCASES TO REACH A CONDITION-COVERAGE THRESHOLD (CHIPFUZZER, TABLE 2).

Method	RocketCore (75%)	BOOM (90%)
ChipFuzzer	645	734
BMCFuzz	4,352 (6.7×)	5,600 (7.6×)
Cascade	5,946 (9.2×)	7,694 (10.5×)
TheHuzz	17,126 (26.6×)	23,005 (31.3×)

TABLE XIII
ENCARSIA BUGS DETECTED (OF 30 PER CORE; CHIPFUZZER, FIG. 8 AND SEC. 5.3).

Fuzzer	Rocket	BOOM	CVA6	Rate (/90)
TheHuzz	10	15	8	36.7%
Cascade	11	15	10	40.0%
BMCFuzz	12	16	8	40.0%
ChipFuzzer (CBG)	20	20	15	61.1%
ChipFuzzer CG only	1	2	2	5.6%
ChipFuzzer BG only	12	12	11	38.9%

IX. ROADMAP TO FPGA ACCELERATION

TurboFuzz shows that the host–FPGA link, not the DUT, limits FPGA-assisted fuzzers, and therefore places generation, execution, checking and coverage collection on one Zynq

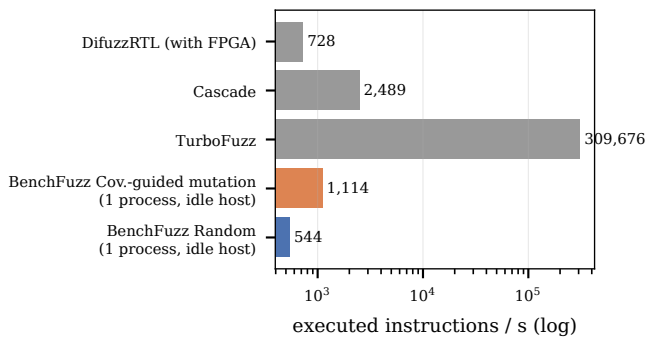


Fig. 9. Executed instructions per second: published (TurboFuzz Table I) and measured for our software loop (one process under load).

UltraScale+ device [4]. We plan the same split in three stages.

Stage A (done): The software loop of this paper, which remains the triage environment: minimisation, localisation and repair need full visibility and are run only on failures.

Stage B: DUT farm in the PL: Yosys `synth_xilinx -family xcup` maps PicoRV32 in our configuration to 1,502 LUTs, 945 flip-flops, 4 DSP48E2 and 6 LUTRAM primitives (pre place-and-route), 0.29% of the LUTs of the XCZU19EG that TurboFuzz used. Each DUT instance gets an RVFI capture FIFO and hardware coverage counters (one bit per Verilator line point that we found reachable is enough, because Sec. VII-C shows the rest is configuration-dead). The PS (Cortex-A53) runs the corpus manager and a native port of the reference model; the PL streams only mismatching retirement windows and coverage bitmaps over AXI.

Stage C: on-chip generation and self-checking: Following TurboFuzz, the generator moves into the PL and the oracle changes from lockstep comparison to self-checking programs (Cascade-style entanglement, or a signature register compared with the PS reference asynchronously), so that the PS is involved only for failures. The host keeps LLM-guided generation of seeds and the repair loop; a repaired RTL needs re-synthesis (minutes to hours), so repair is always validated in software first.

Risks: PS–PL bandwidth and latency, coverage-counter area, the translation of on-chip coverage to the metrics of Sec. IV, and the loss of observability, which is why every failure is replayed in the software loop. No throughput figure is claimed until measured.

X. Threats to Validity

Internal. The generators are our re-implementations of published strategies; tuning them differently could change the ranking. Budgets are counted in programs; seconds were measured with seven processes sharing eight cores. The template-repair baseline uses generic operators, but they were chosen knowing the bug classes of the suite. The held-out set detects overfitting only as far as random programs exercise the patched logic. *Construct.* RTL coverage counts Verilator points, which differ from the condition coverage of the literature; the functional covergroups are our own; C_U depends on weights. Detection means a lockstep mismatch on the RV32IM subset; bugs outside it (CSR, interrupts, compressed ISA) are invisible by construction. *External.* One small in-order core in one configuration; injected bugs rather than real bugs except P6; results may not transfer to out-of-order cores, where coverage does not saturate. *LLM.* No LLM-based result in this paper was re-measured with the current code; the Run C numbers come from one campaign with a free API tier whose model changed during the run.

XI. Conclusion

On PicoRV32 a carefully specified lockstep oracle raises no false alarm on upstream RTL, reproduces a published bug on the same commit and, through a mutational generator, exposes a residual FENCE deviation on current master. Fed by simple random and mutational generators it saturates all reachable RTL within a few hundred programs and finds every observable injected bug but one within a few programs; what makes a bug hard is the specificity of its trigger, not its structural tier, and coverage guidance did not shorten detection. The remaining effort lies after detection. Slicing and delta debugging shrink fuzzer failures to a few instructions, with slicing doing most of the work. Gated repair accepts every correct patch, and a template search without any LLM produces gate-passing patches for most bugs, but two of them are not correct, one functionally wrong for rare operands and one changing an interface the oracle does not observe; only formal certification separated them. We therefore argue that LLM repair results should be reported with formal (or at least interface-complete) certification, not with test-suite pass rates alone. The next steps are LLM pass@k on the suite under the same certification, widening the ISA scope to CSRs so that Cascade P1–P5 and counter faults become observable, a larger out-of-order core where coverage does not saturate, and Stage B of the FPGA roadmap.

Artifact

Code, compact result logs (`results/v0.4`) and the generator of every table and figure: branch `v0.4-evaluation` of https://github.com/HumourousTree/ml-guided-hardware-fuzzing; `scripts/run_full_evaluation.sh` reruns everything and `tools/compare_literature.py` regenerates `paper/generated` and `paper/figures`.

References

[1] K. Wang, C. Li, K. Chang, S. Cai, Y. Han, Y. Wang, "When fuzzing meets understanding: LLM-driven semantic test generation for RTL verification," arXiv:2607.10340, Jul. 2026.

[2] F. Solt, K. Ceesay-Seitz, K. Razavi, "Cascade: CPU fuzzing via intricate program generation," in *USENIX Security*, 2024.

[3] R. Kande *et al.*, "TheHuzz: Instruction fuzzing of processors using golden-reference models for finding software-exploitable vulnerabilities," in *USENIX Security*, 2022.

[4] "TurboFuzz: FPGA accelerated hardware fuzzing for processor agile verification," arXiv:2509.10400v2, Feb. 2026.

[5] "BMCFuzz: Hybrid verification of processors by synergistic integration of bound model checking and fuzzing," IEEE, 2025.

[6] M. Bölcskei, F. Solt, K. Ceesay-Seitz, K. Razavi, "Encarsia: Evaluating CPU fuzzers via automatic bug injection," in *USENIX Security*, 2025.
[7] K. Laeufer *et al.*, "RFUZZ: Coverage-directed fuzz testing of RTL on FPGAs," in *ICCAD*, 2018.
[8] J. Hur *et al.*, "DifuzzRTL: Differential fuzz testing to find CPU bugs," in *IEEE S&P*, 2021.
[9] A. Zeller, R. Hildebrandt, "Simplifying and isolating failure-inducing input," *IEEE TSE*, 28(2), 2002.
[10] G. Misherghi, Z. Su, "HDD: Hierarchical delta debugging," in *ICSE*, 2006.
[11] R. Abreu, P. Zoeteweij, A. van Gemund, "On the accuracy of spectrum-based fault localization," in *TAIC PART*, 2007.
[12] H. Ahmad, Y. Huang, W. Weimer, "CirFix: Automatically repairing defects in hardware design code," in *ASPLOS*, 2022.
[13] M. Chen *et al.*, "Evaluating large language models trained on code," arXiv:2107.03374, 2021.
[14] C. Wolf, "riscv-formal: RISC-V formal verification framework," https://github.com/YosysHQ/riscv-formal.

APPENDIX A
REFINEMENTS TO THE EVALUATION BRIEF

- *Names.* The tool is ChipFuzzer (not "ChipFuzz"); ChipFuzzer and TurboFuzz are preprints, not peer-reviewed.
- *24-hour condition coverage on RocketCore/BOOM/CVA6 and Encarsia rates* are quoted (Tables XI–XIII) but cannot be cross-referenced numerically with PicoRV32 telemetry; we compare methods on one harness instead.
- *AND→OR in* `picorv32_pcpi_fast_mul`*:* the module has only logical ANDs; the literal mutation is equivalent on every valid program and observable only through illegal encodings (Sec. VII-D); an observable signedness fault in the same module and a bitwise AND→OR in the ALU are used.
- *Stuck-at on low* `mem_addr` *bits:* bits 1:0 are always zero under the word-aligned convention, so bit 2 is used.
- `mem_ready` *drops* are injected in the core's reaction to the testbench input (deterministic and intermittent variants).
- *Uncleared interrupt trap flags* are unobservable with `ENABLE_IRQ=0`; an uncleared `latched_branch` substitutes.
- *C++ ISS gcov* does not apply (Python reference); statement coverage is measured with a line tracer.
- *pass@k* requires independent samples; the feedback loop with three lint retries is reported separately.
- *"Hierarchical" delta debugging* was flat ddmin in v0.3; HDD over basic blocks and instructions is now implemented and compared with ddmin.

APPENDIX B
REPRODUCIBILITY

```
python3 tools/fetch_upstream.py -commit ef203c2
bash scripts/run_full_evaluation.sh 7
python3 -m harness.template_repair -budget 30
python tools/compare_literature.py
```